MIT | Academy of Engineering

(An Autonomous Institute affiliated to Savitribai Phule Pune University)

Heap Sort

Name: Ananya Jadhav

Roll No. : 62 PRN: 202401040097

Batch : A4

Code:

```
#include <iostream>
#include <algorithm> // Required for swap() in some C++ versions

using namespace std;

void heapify(int arr[], int n, int i)
{
    int largest = i;  // Initialize largest as root
    int l = 2 * i + 1;  // left child index
    int r = 2 * i + 2;  // right child index

    // If left child is larger than root
    if (l < n && arr[l] > arr[largest])
```

```
        largest = l;

    // If right child is larger than largest so far
    if (r < n && arr[r] > arr[largest])
        largest = r;

    // If largest is not root
    if (largest != i)
    {
        swap(arr[i], arr[largest]);

        // Recursively heapify the affected sub-tree
        heapify(arr, n, largest);
    }
}

// main function to do heap sort
void heapSort(int arr[], int n)
{
    // Step 1: Build heap (rearrange array)
    // We start from the last non-leaf node and work upwards
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);

    // Step 2: One by one extract an element from heap
    for (int i = n - 1; i > 0; i--)
    {
```

```
        // Move current root (largest) to the end of the array
        swap(arr[0], arr[i]);

        // call max heapify on the reduced heap to restore heap property
        heapify(arr, i, 0);
    }
}

/* A utility function to print array of size n */
void printArray(int arr[], int n)
{
    for (int i = 0; i < n; ++i)
        cout << arr[i] << " ";
    cout << "\n";
}

int main()
{
    int n, i;
    int list[30];

    cout << "Enter number of elements (max 30): ";
    cin >> n;

    cout << "Enter " << n << " numbers: ";
    for(i = 0; i < n; i++)
        cin >> list[i];
```

```
    heapSort(list, n);

    cout << "Sorted array is: \n";
    printArray(list, n);

    return 0;
}
```

Output:

```
Enter number of elements (max 30): 5
Enter 5 numbers: 1 6 8 9 4
Sorted array is:
1 4 6 8 9
```

```
Enter number of elements (max 30): 6
Enter 6 numbers: 55 88 44 33 66 75
Sorted array is:
33 44 55 66 75 88
```

```
Enter number of elements (max 30): 5
Enter 5 numbers: 94 66 11 24 54
Sorted array is:
11 24 54 66 94
```

```
Enter number of elements (max 30): 6
Enter 6 numbers: 54 21 87 62 45 12
Sorted array is:
12 21 45 54 62 87
```